habiPEt | Architecture and Protocols

MTM4186 - Mobile Programming | Individual Milestone Submission | Proposed Technical Design

1. Project Overview

habiPEt is a gamified habit-tracking mobile application. Users create habits, record daily completion, monitor progress, earn milestone rewards, and use earned rewards to feed and develop a virtual pet. This document describes the planned architecture for the milestone; it does not claim that every feature has been implemented.

2. Architectural Approach

The application will be developed in Flutter (Dart) using a feature-based structure with a Controller-Repository pattern. Each feature owns its screens, state and business logic, and data-access abstractions. This separation improves maintainability, testing, and future expansion.

Presentation Layer	**Flutter screens / widgets / Material 3 navigation**
Controller & Domain Logic	**HabitController, PetController, ProgressController; validation and reward rules**
Repository / Persistence	**Repository contracts + a local storage implementation (to be selected)**

Dependency direction: UI calls controllers; controllers depend on repository contracts; concrete storage implementations fulfill those contracts. Widgets should not access persistence directly.

3. Main Feature Modules

Module	Primary responsibility
Habit Management	Create, edit, complete, archive and list active habits.
Progress & Calendar	Store completion history; calculate streaks and daily/period progress.
Achievements	Evaluate 7-, 14- and 21-day milestones and track awarded badges.
Virtual Pet	Maintain pet state; spend earned rewards to feed the pet; update level based on feeding.
Profile & Settings	Manage preferences, including theme selection and basic profile information.

4. Proposed Data Models

Habit: id, title, category, schedule, isArchived. HabitCompletion: habitId, date, completedAt. RewardBalance: availablePoints, earned/spent transactions. Pet: ageStartDate, feedingProgress, level. Badge: badgeId, achievedAt, status. UserPreferences: themeMode and optional local settings.

Technical Design | Data Flow and Interfaces

habiPEt - Individual Milestone Submission

5. Core Interaction and Data Flow

Use case A - Complete a daily habit:

- The user taps the completion control on a habit card.
- HabitController checks whether that habit is scheduled and already completed for the day.
- The completion is saved through HabitRepository; duplicate daily rewards are prevented.
- Progress logic recalculates completion counts and streaks.
- Achievement logic checks whether a milestone has been reached and records any new badge/reward.
- The UI refreshes the completed state, progress indicators, and available reward balance.

Use case B - Feed the virtual pet:

- The user opens the Pet screen and selects Feed.
- PetController verifies sufficient available rewards.
- The reward balance is reduced and the feeding event is persisted consistently.
- Feeding progress is updated; the pet level increases only when the defined feeding threshold is reached.
- The interface displays the new balance, feeding progress, and pet state.

Important rule: Pet age reflects elapsed time since the pet was created. Pet level reflects feeding progress. Habit completion can earn resources for feeding, but it does not directly increase pet level.

6. Repository Interfaces / Contracts

Contract (proposed)	Representative operations
HabitRepository	getActiveHabits(), createHabit(), updateHabit(), archiveHabit(), recordCompletion()
ProgressRepository	getCompletions(range), getStreak(habitId)
RewardRepository	getBalance(), grantRewardOnce(), spendReward()
PetRepository	getPet(), saveFeedingEvent(), updatePet()
BadgeRepository	getBadges(), awardBadgeOnce()
SettingsRepository	loadPreferences(), savePreferences()

These are design-level contracts, not assertions about existing method signatures. Exact types and method names may change during implementation.

7. Protocols, Interfaces and Communication

- Internal communication: Dart method calls and controller state updates; the UI observes state changes and rebuilds relevant widgets.
- Persistence interface: Repository contracts isolate the app from the selected on-device database or key-value storage solution.
- Platform interface: Flutter framework widgets and platform abstractions provide a consistent Android/iOS UI.
- Network protocols: The milestone architecture is local-first and does not require a remote server, HTTP API, or authentication service. Cloud sync could be added later, but is out of current scope.

Technical Design | Quality and Scope

8. Persistence and Data Integrity

The initial application is planned as local-first. A concrete persistence package has not yet been finalized; the repository layer allows that choice to be made without coupling storage details to screens. Structured records (habits, completions, rewards, and pet events) should be stored persistently across app restarts.

- Avoid counting a habit as completed more than once on the same scheduled date.
- Make reward granting and spending idempotent where appropriate to prevent duplicate points or badges.
- Keep the reward deduction and pet-feeding update consistent, ideally within one transactional operation if the chosen storage supports it.
- Recalculate derived values such as streaks from recorded completion history when feasible.

9. State Management and Navigation

Controllers will coordinate user actions and expose view state, including loading, success, and error conditions. The UI will display habit lists, calendar views, pet progression, badges, and settings based on that state. The final Flutter state-management mechanism will be selected during implementation rather than presented as already integrated.

The planned navigation begins with a Welcome/Onboarding screen for first-time use, followed by a Home Dashboard. From Home, users can open Add Habit, Pet, Progress, Achievements, and Profile/Settings. Archive management remains accessible from habit management or settings.

10. Testing and Accessibility Considerations

- Logic: test completion rules, duplicate prevention, streak boundaries, milestones, reward balance, and level thresholds.
- UI: test primary flows such as add habit, mark complete, feed pet, and browse progress on common phone sizes.
- Accessibility: readable typography, semantic labels, adequate touch targets, sufficient contrast, and non-color-only status indicators.

11. Milestone Scope and Future Work

Milestone deliverables: design justification, user-flow PDF, Figma wireframes, Figma high-fidelity mockups, an interactive Figma prototype, and this architecture document. The milestone establishes the intended mobile experience and technical direction; fully functioning production features are not assumed.

Possible later extensions: scheduled notifications, optional cloud backup/synchronization, more detailed analytics, and deeper pet customization. These are future options and not part of the current implementation commitment.

12. Summary

The proposed feature-based Controller-Repository architecture separates presentation, business rules, and persistence. It supports habit tracking, reliable progress evaluation, milestone-based rewards, and a virtual pet whose growth follows feeding activities. Local-first storage and clearly defined repository contracts keep the project achievable for an individual developer while leaving room for later enhancements.